

VLM코드 정리

Qwen2.5-VL 모델을 사용해서 이미지 속 인물의 성별, 나이대, 상의, 하의, 모자 정보를 텍스트로 추출하는 코드 PersonAnalyzer 클래스로 모듈화한 코드입니다

1. 전체 코드

```
import time
import torch
from PIL import Image
from transformers import (
    AutoProcessor,
    Qwen2_5_VLForConditionalGeneration,
    BitsAndBytesConfig,
)

class PersonAnalyzer:
    def __init__(
        self,
        model_id="Qwen/Qwen2.5-VL-3B-Instruct",
        min_pixels=128 * 28 * 28,
        max_pixels=384 * 384,
        use_4bit=True,
    ):
        self.model_id = model_id

        print("모델 로딩 중...")
        start = time.time()

        self.processor = AutoProcessor.from_pretrained(
            self.model_id,
            min_pixels=min_pixels,
            max_pixels=max_pixels,
        )

        if use_4bit:
            quantization_config = BitsAndBytesConfig(
                load_in_4bit=True,
                bnb_4bit_compute_dtype=torch.float16,
                bnb_4bit_quant_type="nf4",
                bnb_4bit_use_double_quant=True,
            )
```

```

        self.model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
            self.model_id,
            quantization_config=quantization_config,
            device_map="auto",
        )
    else:
        self.model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
            self.model_id,
            torch_dtype=torch.float16,
            device_map="auto",
        )

```

```

end = time.time()
print("모델 로딩 완료")
print(f"모델 로딩 시간: {end - start:.2f}초")
print(self.model.hf_device_map)

```

```
def _build_messages(self, image):
```

```
    return [
```

```
        {
```

```
            "role": "user",
```

```
            "content": [
```

```
                {"type": "image", "image": image},
```

```
                {
```

```
                    "type": "text",
```

```
                    "text": ""
```

```
                        너는 지능형 CCTV 프로그램이야.
```

```
                        이미지 속 인물의 성별, 추정 나이대, 상의, 하의, 모자만 추출
```

해.

```
                        규칙:
```

```
                        - 한국어로만 출력
```

```
                        - 문장으로 설명하지 말 것
```

```
                        - '~입고 있음', '~쓰고 있음', '~입니다' 같은 서술어 금지
```

```
                        - 각 항목은 명사형으로만 출력
```

```
                        - 의상은 색상을 포함해서 출력
```

```
                        - 모자가 보이지 않으면 '모자: 없음' 출력
```

```
                        출력 형식은 반드시 아래 형식만 사용:
```

```
                        성별:
```

```
                        나이대:
```

```
                        상의:
```

```
                        하의:
```

```
                        모자:
```

```
                        "" ,
```

```
                    },
```

```

        ],
    }
]

def analyze(self, image_or_path, max_new_tokens=50, show_time=True):
    if isinstance(image_or_path, str):
        image = Image.open(image_or_path).convert("RGB")
    else:
        image = image_or_path.convert("RGB")

    messages = self._build_messages(image)

    inputs = self.processor.apply_chat_template(
        messages,
        tokenize=True,
        add_generation_prompt=True,
        return_dict=True,
        return_tensors="pt",
    ).to(self.model.device)

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    if show_time:
        print("이미지 분석 중...")

    infer_start = time.time()

    with torch.no_grad():
        generated_ids = self.model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
        )

    infer_end = time.time()

    generated_ids_trimmed = generated_ids[:,
inputs["input_ids"].shape[1]:]

    result = self.processor.batch_decode(
        generated_ids_trimmed,
        skip_special_tokens=True,
        clean_up_tokenization_spaces=False,
    )[0]

```

```

        result = self._clean_result(result)

    if show_time:
        print(f"이미지 분석 시간: {infer_end - infer_start:.2f}초")

    return result

def _clean_result(self, text):
    text = text.strip()
    text = text.replace("입니다", "")
    text = text.replace("를", "")
    text = text.replace("을", "")
    text = text.replace("입고 있습니다", "")
    text = text.replace("쓰고 있습니다", "")
    text = text.replace(".", "")
    return text

if __name__ == "__main__":
    analyzer = PersonAnalyzer()

    result = analyzer.analyze("person.jpg")

    print("\n==== 분석 결과 =====")
    print(result)

```

2. 코드 역할 설명

어느정도로 자세하게 적어드려야할지 감이 안잡혀서 일단 큰 틀로 정리해드릴게요.

import하고 from코드는 단순히 모듈 불러오는 코드입니다.

PersonAnalyzer라는 클래스를 만들어서 외부 파일에서 `analyzer = PersonAnalyzer()` 클래스 불러오고, `result = analyzer.analyze("person.jpg")` 로 객체 생성해서 이미지 분석 가능합니다.

`__init__()` 역할

- Qwen 다운로드/로딩 - 현재 저희가 쓰는 VLM모델
- 이미지 크기 제한 - 이미지 크기가 크면 VRAM부족해져서 뱉납니다.
- 4bit양자화 사용 - Qwen모델 자체가 워낙 커서 VRAM아끼기 위해 AI모델 압축했다고 생각하시면 돼요.
- processor라는 AI입력 변환기(?)같은걸 사용해서 이미지, 문자를 ai가 이해할 수 있는 숫자 텐서로 변환 - 저도 솔직히 그냥 아, 그렇구나. 하고 넘겼습니다.
- GPU자동 배치
- 그외 테스트용 로딩시간 확인 코드

`_build_messages()` 역할

- VLM에게 역할부여하고 양식 지정하는 코드 - 그냥 제미나이나 지피티 프롬프트창 생각하시면 되요.
- 참고로 앞에 `_`가 붙는 코드는 `private`코드로 사용한다는 뜻입니다.
- 실제 분석은 여기서 수행하지 않고, 분석에 사용할 질문 양식만 만드는 역할
- 쉽게 정리하면 분석할 이미지와 텍스트 프롬프트를 묶어서 반환하는 코드입니다.

`analyze()` 역할

- 애가 핵심인데 한줄로 정리하면 '입력받은 이미지를 실제로 AI에 넣어서 분석시키는 함수'입니다.
- AI가 출력하는 최대 토큰 제한 - 길게 출력할수록 gpu낭비 심해서 제한한겁니다.
- 이미지 형식 RGB로 통일 - RGBA처럼 다른 이미지 형식일때 RGB로 변환.
- `_build_messages()` 함수 호출 - 위에 설명한 함수입니다. 이미지, 텍스트 프롬프트 묶어서 메시지 생성.
- `processor`로 이미지와 메시지 AI가 이해 가능한 형태로 변환(이미지, 글자 -> 숫자 텐서)
- Qwen 모델의 `generate()`를 통해 이미지 분석 수행
- 입력 데이터를 GPU로 이동 - 모델이 GPU에 올라가있기 때문에
- 안 쓰는 GPU 메모리 정리 - VRAM부족 방지용
- AI 출력에서 입력 프롬프트 부분 제거 - 가끔 입력 프롬프트도 같이 출력되는거 방지
- AI가 출력하는 숫자 텐서를 문자열로 변환
- 서술형 문장 뒷부분 컷팅

2. 외부 파일 입력 예시

```
# main.py일때
from vlm_person_analyzer_qwen import PersonAnalyzer

analyzer = PersonAnalyzer()
result1 = analyzer.analyze("1.jpg")
result2 = analyzer.analyze("2.jpg")
```

3. 출력 예시

```
모델 로딩 중...
모델 로딩 완료
모델 로딩 시간: 15.32초
이미지 분석 중...
이미지 분석 시간: 6.21초

===== 분석 결과 =====
성별: 남성
나이대: 20대
```

상의: 회색 반팔 티셔츠

하의: 청바지

모자: 없음